

FlickDom (플릭덤)

상세 게임 설명 및 프로그램 구현 계획서

프로젝트명	FlickDom (Freedom 기반 물리 점령 게임)
작성일자	2026년 6월 30일
버전	V1.5 (상세 연출 및 R&R 적용본)
개발 팀 구성	클라이언트 A : 임찬진 (물리, 카메라, 입력)
	클라이언트 B : 박신우 (로직, 분할화면, 데이터)
	모델 / 이펙트 : 황인성 (아트 리소스, VFX)

1. 게임 상세 설명 (Game Description)

1.1. 게임 컨셉 및 테마

FlickDom(플릭덤)은 자유를 뜻하는 'Freedom'에서 착안한 타이틀로, 플레이어의 정교한 물리적 조작감과 보드판 위의 치밀한 공간 점령 전략이 결합된 대전형 보드게임입니다. 단순히 오브젝트를 멀리 보내거나 상대를 밀어내는 기존의 알까기 문법을 넘어, 5x5 격자 공간 위에 자신의 지배력을 구축하고 난이도별 점령 카드를 선점해야 하는 고도의 심리 전략을 핵심 재미 요소로 삼습니다.

1.2. 게임 구성 요소 사양 (Components)

- **게임 플레이 보드:** 물리적인 오브젝트가 부딪히고 굴러가는 5x5 형태의 정사각형 필드입니다. 외곽은 토큰이 튕겨 나갈 수 있는 물리적인 벽(Wall Boundary)으로 둘러싸여 있습니다.
- **토큰 맵 (Token Map):** 플레이 보드와 별개로 존재하는 5x5 크기의 논리적 점령판입니다. 플레이어가 획득한 영토 소유권이 영구적으로 시각화되는 공간입니다.
- **지역 지배 토큰:** 각 플레이어별로 5개씩, 총 10개가 제공됩니다. 토큰 맵에 배치하여 해당 격자의 소유권을 표시하는 용도입니다.
- **플레이 토큰 (물리 객체):** 색상별로 3개씩, 총 6개가 존재합니다. 플레이어가 턴 중에 직접 손가락으로 튕겨 이동시키는 물리 연산의 주체입니다. 라운드가 끝나면 수거되어 다시 시작 지역으로 재배치됩니다.
- **지역 점령 카드:** 게임의 승리 조건을 형성하는 카드 총 9장으로 구성됩니다.
 - **EASY 카드 (2장):** 보드 내 2칸의 격자 패턴 점령 요구
 - **NORMAL 카드 (3장):** 보드 내 3칸의 격자 패턴 점령 요구
 - **HARD 카드 (4장):** 보드 내 4칸의 격자 패턴 점령 요구

1.3. 게임 상세 규칙 및 예외 처리 조건

1.3.1. 단계별 진행 및 카드 획득

게임은 EASY → NORMAL → HARD 순서로 총 3단계에 걸쳐 진행됩니다. 이전 단계의 모든 지역 점령 카드를 플레이어들이 나누어 획득하여 필드에 남은 카드가 없어야만 다음 난이도 단계의 카드가 중앙 상단에 오픈됩니다. 전체 3번의 게임이 완전히 종료된 시점에 누적된 지역 점령 카드의 개수가 가장 많은 플레이어가 최종 승리합니다.

1.3.2. 객체 이동 및 물리 제약 규칙 (중요 예외 처리)

플레이어는 자신의 차례에 시작 지역 내 원하는 곳에 3개의 플레이 토큰을 자유롭게 배치한 후, 순차적으로 한 개씩 선택하여 튕깁니다. 이때 튕겨 나간 토큰은 반드시 외곽 벽, 상대방의 토큰, 또는 이미 배치된 자신의 다른 토큰 중 최소 하나 이상에 무조건 충돌해야 합니다. 만약 어떤 오브젝트와도 부딪히지 않고 공허하게 빈 공간에 멈춰 섰을 경우, 물리적 조작 실패로 간주하여 해당 토큰은 라운드 연산에서 즉시 제외됩니다. 또한, 충돌 과정 중 상대방의 강한 타격으로 인해 플레이 보드 영역 밖으로 밀려 나간 객체 역시 해당 라운드 배치 판정에서 영구 제외됩니다.

1.3.3. 토큰 배치 및 소유권 탈취 (Interaction)

라운드가 종료되면 모든 토큰의 물리 속도가 멈춘 상태에서 선공부터 차례대로 판정이 시작됩니다. 플레이 보드 위에 살아남은 플레이 토큰의 최종 위치를 기반으로 우측 토큰 맵에 자신의 '지역 지배 토큰'을 배치합니다. 토큰이 완벽히 한 칸에 들어가지 않고 다수의 격자 영역 경계면에 걸쳐 있을 경우, 플레이어는 팝업 UI를 통해 걸쳐 있는 칸 중 원하는 단 한 칸을 선택하여 지배권을 행사할 수 있습니다.

선택한 칸에 이미 상대방의 지역 지배 토큰이 놓여 있는 상태라 하더라도, 해당 칸을 강제로 빼앗아 자신의 토큰으로 교체하고 상대방의 토큰은 소유자에게 돌려보냅니다. 만약 플레이어가 가진 5개의 지역 지배 토큰이 이미 맵에 모두 깔려 있는 상황이라면, 보드판 위에 새로 생긴 위치로 기존에 맵에 있던 토큰 중 하나를 선택해 이동시킬 수 있습니다. 배치가 끝난 직후 상단 카드 조건 모양과 일치하는 격자 점령 형태를 갖추었다면 즉시 카드를 획득하며, 동시에 여러 카드의 조건을 만족했다면 중복 획득이 가능합니다.

1.3.4. 차기 라운드 선·후공 결정 우선순위

매 라운드가 종료될 때마다 다음 3가지 우선순위에 의거하여 철저하게 연산되어 결정됩니다.

1. **우선순위 1:** 현재까지 누적된 지역 점령 카드를 더 많이 획득한 플레이어가 선공을 가져갑니다.
2. **우선순위 2:** 획득 카드 수가 동일할 경우, 토큰 맵에 배치되어 있는 자신의 지역 지배 토큰 개수가 더 많은 플레이어가 선공이 됩니다.
3. **우선순위 3:** 배치된 토큰 개수마저 완벽히 동일하다면, 직전 라운드에서 후공이었던 플레이어가 우선권을 넘겨받아 선공이 됩니다.

2. 화면 구성 및 카메라 연출 계획 (View & Camera System)

2.1. 대기 및 배치 상태: 2D 분할 화면 (Split-Screen 2D)

게임의 기본 디스플레이 모드는 화면이 반으로 정밀하게 분할된 2D 탑다운 뷰입니다. 좌측(혹은 상단)에는 물리 기반의 '게임 플레이 보드'가, 우측(혹은 하단)에는 영토 현황을 나타내는 '토큰 맵'이 동시에 렌더링되어 플레이어가 물리적 전황과 논리적 점령 상태를 한눈에 파악하며 전략을 구상할 수 있도록 돕습니다.

상시 노출 UI 제약: 화면의 정중앙 최상단 영역에는 현재 단계에서 달성해야 하는 '지역 점령 카드' UI가 항상 고정 배치됩니다. 이 UI는 카메라 뷰가 어떻게 변화하든 화면 레이어의 최상위에 위치하여 렌더링 오버레이를 유지해야 합니다.

2.2. 차례 진행 상태: 3D 45도 전체 화면 (Full-Screen 3D Action)

특정 플레이어의 턴이 시작되어 조작 권한이 활성화되는 순간, 2D 분할 화면은 부드러운 카메라 블렌딩 연출을 거쳐 '게임 플레이 보드' 중심의 전체 화면으로 통합됩니다. 이때 카메라는 기존의 수직 직각(90도) 탑다운 뷰에서 **3D 형식의 45도 사각 뷰(Isometric Perspective 45°)**로 다이내믹하게 회전 및 전진합니다. 이를 통해 알짜기 조작 시 오브젝트의 입체감과 벽면의 높낮이, 정확한 드래그 벡터의 궤적을 시각적으로 극대화하여 손맛을 높입니다.

2.3. 화면 복귀 및 시퀀스 흐름

플레이어가 배정된 3개의 플레이 토큰을 모두 튕겨 물리적인 시뮬레이션 연산이 완전히 완료되면, 카메라는 다시 부드럽게 뒤로 빠지며 원경 90도 탑다운 상태의 **2D 분할 화면(반반 뷰)**으로 전환됩니다. 플레이어는 복귀된 화면을 통해 자신이 튕긴 돌들의 최종 위치를 확인하고, 토큰 맵에 지배 토큰을 배치하는 '배치 화면 상호작용'을 수행하게 됩니다.

3. 프로그램 구현 상세 계획 (Technical Implementation Plan)

3.1. 유한 상태 머신 (FSM) 기반 코어 게임 루프 설계

물리 연산의 비동기적 특성과 턴제 보드게임의 논리적 순차성을 완벽하게 동기화하기 위해, 전체 시스템 흐름을 `GameModeManager` 구조 내에서 유한 상태 머신으로 통제합니다.

게임 상태 (State)	주요 처리 로직 (Logic)	화면 연출 및 카메라 뷰
1. Ready State	라운드 초기화, 선후공 판단 알고리즘 작동, 3개의 플레이 토큰을 시작 지역에 로드 및 대기	2D 분할 화면 (90° Top-Down)
2. Flicking State (알까기)	사용자 마우스/터치 드래그 입력 접수, 궤적 라인 렌더링, 토큰 순차 발사(총 3회 제한), 충돌 여부 체크 플래그 활성화	3D 전체 화면 (45° Quarter-View)
3. Physics Processing	실시간 물리 월드 연산 수행. 모든 발사 토큰의 <code>Rigidbody2D.velocity.magnitude</code> 스칼라 값이 0에 수렴할 때까지 유저 입력 원천 차단	3D 전체 화면 유지
4. Placement State (배치)	생존 토큰 위치 매핑 계산. 다중 격자 충돌 시 선택 UI 팝업. 토큰 맵 데이터 갱신 및 소유권 강제 탈취/반환 로직 수행	2D 분할 화면 복귀
5. Card Match State	ScriptableObject 패턴 데이터와 <code>int[5,5]</code> 보드 데이터 행렬 연산 비교. 카드 매칭 성공 시 획득 처리 및 단계 승급 조건 검사	2D 분할 화면 (상단 카드 이펙트)

3.2. 핵심 로직 흐름 (Core Gameplay Flow)

시스템의 백엔드 흐름은 다음 3단계 코어 파이프라인을 엄격하게 준수하여 아키텍처 간의 간섭을 최소화합니다.

```
// 핵심 시퀀스 가상 흐름 정의
void ExecuteCoreLoop() {
    // 1단계: 알까기 가동 및 물리 처리 완료 대기
    StartCoroutine(FlickingPhaseRoutine(() => {
        // 2단계: 화면 전환 후 월드 좌표 기반 5x5 그리드 배치 처리
```

```

        ChangeCameraToSplitView();
        GridMappingSystem.ResolveTokenPositions();

        // 3단계: 갱신된 데이터를 바탕으로 상단 카드 상호작용 검사
        CardInteractionSystem.CheckPatternMatching();
    }));
}

```

3.3. 물리-그리드 좌표 매핑 및 판정 알고리즘

3D 45도 뷰 공간 상에서 최종 정지한 물리적 오브젝트의 `transform.position` 값은 실수형(float) 월드 좌표계입니다. 이를 논리 데이터인 `int[5,5]` 배열 인덱스로 치환하기 위해 아래의 매핑 수학 공식을 적용합니다.

보드의 중심을 (0,0)으로 잡고, 타일 하나의 가로세로 크기를 `cellSize`라 정의할 때, 인덱스 계산식은 다음과 같습니다.

```

int gridX = Mathf.FloorToInt((worldPos.x - boardOriginX) / cellSize);
int gridY = Mathf.FloorToInt((worldPos.z - boardOriginZ) / cellSize);

```

만약 토큰의 Collider가 셀의 경계 영역에 중첩되어 물리적 충돌구 외곽이 다수의 타일에 닿아 있을 경우, `Physics2D.OverlapCircleAll` 메서드로 검출된 모든 타일 인덱스 리스트를 추출한 뒤, 클라이언트 B의 UI 시스템으로 이벤트를 전송하여 플레이어 선택 분기 처리를 유도합니다.

4. 팀원별 상세 업무 분담 (R&R)

4.1. 클라이언트 프로그래머 A : 임찬진

담당 영역: 물리 엔진 코어, 카메라 연출, 인게임 조작 시스템

- **입력 및 조작:** 마우스 클릭 및 드래그 거리에 비례한 튕기기 힘(\$Force\$) 벡터 연산 시스템 구축, 조준 방향 및 예상 궤적을 실시간으로 시각화하는 `LineRenderer` 가이드 시스템 구현.
- **카메라 제어:** 유니티 `Cinemachine` 패키지를 도입하여 2D 분할 화면 뷰와 3D 45도 쿼터뷰 간의 가상 카메라(Virtual Camera) 블렌딩 트랜지션 연출 하이 레벨 스크립팅.
- **물리 충돌 제약 조건:** 각 플레이 토큰에 충돌 감지 클래스를 부착하여, 외곽 벽면 및 타 오브젝트와의 트리거 감지 로직 작성. `bool hasCollided` 상태를 관리하여 미충돌 객체 자동 소멸 스크립트 작성.
- **좌표 연산:** 정지된 오브젝트의 월드 좌표를 추출하여 정수형 그리드 인덱스로 변환해 클라이언트 B의 데이터 매니저로 던져주는 인터페이스 설계.

4.2. 클라이언트 프로그래머 B : 박신우

담당 영역: 시스템 아키텍처, 데이터 모델, UI/UX 프레임워크

- **FSM 매니저 개발:** 전체 게임의 5단계 상태 흐름을 제어하는 상태 패턴 기반 `GameModeManager` 설계 및 중앙 루프 제어.
- **화면 분할 및 UI 구축:** 두 개의 독립된 카메라 화면을 스크린 상에 반반 배치하기 위한 `Viewport Rect` 핸들링 및 상단 고정 점령 카드 레이어 캔버스 구조 최적화.
- **토큰 맵 데이터 관리:** 5x5 크기의 2차원 정수 배열(`int[5,5]`)을 통한 영토 데이터 관리. 상대방 토큰 점유 칸 타격 시 데이터 스와프 및 기존 배치 토큰 회수/재배치 예외 처리 구현.
- **카드 패턴 매칭 알고리즘:** `ScriptableObject` 형식으로 구축된 난이도별(Easy/Normal/Hard) 정적 패턴 행렬 구조와 현재 실시간 5x5 토큰 맵 행렬 데이터를 비트 연산 또는 반복문으로 비교 검사하여 카드 획득 여부를 판정하는 코어 알고리즘 작성. 선/후공 정렬 룰 연산 구현.

4.3. 모델 / 이펙트 디자이너 : 황인성

담당 영역: 2D/3D 그래픽 리소스, VFX 기술 연출, 시각 피드백

- **오브젝트 3D/2D 모델링:** 3D 45도 뷰에서 입체감이 돋보이는 5x5 플레이 보드 프레임, 플레이 토큰 6종(색상별 개별 메쉬 작업), 5x5 토큰 맵 패널 및 지역 지배 토큰 10개 모델링 및 텍스처링 에셋화.
- **UI 및 카드 디자인:** EASY(2칸), NORMAL(3칸), HARD(4칸) 조건 격자 모양이 시각적으로 명확하게 들어오는 카드 레이아웃 아트워크 및 텍스처 리소스 제작.
- **VFX(이펙트) 제작:** 토큰을 드래그할 때 기를 모으는 듯한 인디케이터 이펙트, 오브젝트 충돌 시 불꽃 혹은 파편이 튀는 충돌 효과, 토큰 맵 점령 성공 및 카드 매칭 달성 시 화면 전체를 꾸며주는 화려한 칭호/축하 파티클 연출 구현.
- **애니메이션 셋업:** 카메라 뷰 전환 시 몰입감을 더해줄 화면 페이드인/아웃 연출용 UI 모션 가동.